

TO [Stakeholder Distribution List]

FROM [Your Name]

DATE [Date]

ATTACHMENT Solution Proposal — ML Training Data Ingestion (incl. architecture diagram)

Subject: Proposal — Data Ingestion Approach for ML Training Pipeline

Hi all,

Following our recent discussions on the ML training workflow, I've put together a refined proposal for the data ingestion approach. A summary is below for your review and feedback; the full document, including the architecture diagram, is attached.

Context

- Model retraining cadence: **2–3 times per year**
- Training dataset: high-volume, sourced from **CMS (system of record)**
- Goal: align the ingestion design with MLOps best practices and AWS Well-Architected principles

Key Shifts from the Original Approach

- Move away from a **Spring Batch scheduler** — an always-on service is not justified for a 2–3x/year workload
- Source training data directly from **CMS**, not from the Classify (inference) Service — avoids model-on-model feedback loops
- Replace per-*ApplicationID* REST calls with **time-sliced bulk extracts** — eliminates unnecessary load on the inference path and dramatically reduces ingestion time
- Build ingestion as a **standalone utility project**, not coupled to a service runtime
- Position ingestion as the **first stage of the SageMaker Pipeline**, enabling end-to-end orchestration, lineage, and reproducibility

Proposed Design at a Glance

- On-demand containerized ingestion task — **SageMaker Processing Step** preferred, ECS Fargate as fallback
- Time-sliced bulk extraction from CMS into S3 as partitioned Parquet
- Per-run manifest for auditability, idempotency, and resumable retries
- Outputs partitioned under the pipeline execution ID for byte-level traceability of every trained model

Open Items Requiring Stakeholder Input

- **CMS bulk extraction interface** — confirmation needed on whether a bulk, CDC, or time-windowed extract path exists or can be provisioned

- **Label provenance** — whether ground-truth labels reside in CMS or in a separate adjudication system
- **Schema evolution contract** — agreement on a versioning mechanism between training cycles
- **PII handling boundary** — whether tokenization happens at extract time or during preprocessing
- **Volume sizing** — a short spike on a representative time slice to validate compute sizing

Recommended Next Steps

- Engage the CMS team to confirm bulk extraction options and agree on an interface contract
- Run a volume and runtime sizing spike on one representative time slice
- Draft the SageMaker Pipeline definition with ingestion as Step 1
- Build the ingestion container as a standalone repository
- Validate the full pipeline end-to-end on a single time slice before scaling to a historical backfill

Please share your feedback, concerns, or questions by **[DATE]**, or let me know if a brief working session would help to walk through the design. Happy to discuss any of the above in more detail.

Thanks,

[Your Name]

[Title / Team]